

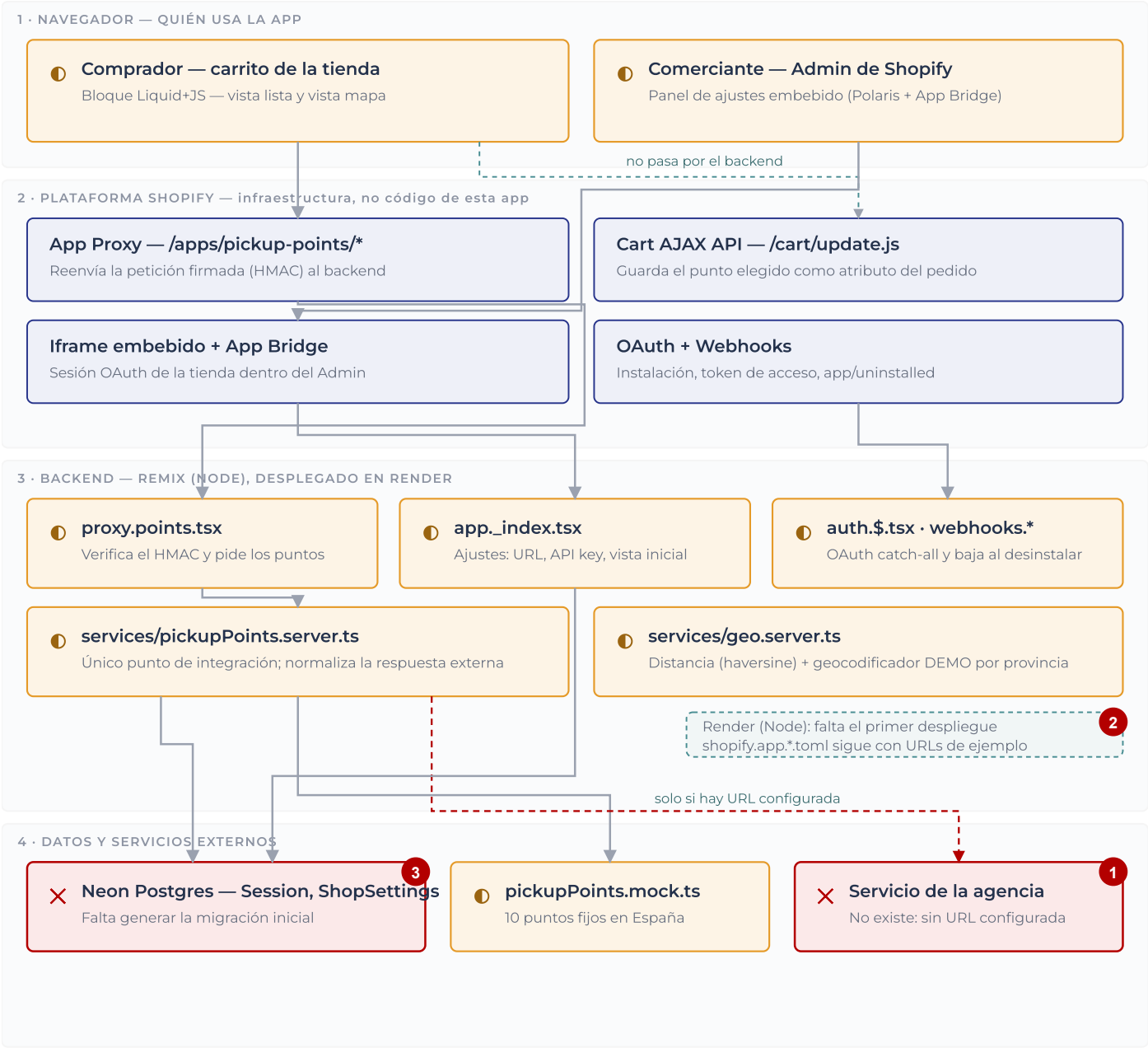
APP DE SHOPIFY · ARQUITECTURA TÉCNICA

El carrito ya sabe pedir puntos de recogida; falta enchufarlo a un transportista real y a un backend desplegado

Los cuatro componentes del MVP están escritos y se hablan entre sí tal y como se explica en este documento, pero hoy corren con datos de demostración: no hay migración de base de datos generada, el backend no está desplegado (Render sigue con URLs de ejemplo) y el servicio web del transportista todavía no existe.

15 de septiembre de 2026 · lectura del repositorio (commits `cc51afe` y `5978c98`) y de la configuración de Shopify Partners — no hay entorno desplegado ni pruebas de extremo a extremo ejecutadas para este análisis.

- hecho, sin verificar en ejecución
- bloqueado: falta algo que no depende de este código
- infraestructura de Shopify (no es código propio)



Los números en rojo remiten a la ficha de cada bloqueo en la página 3. Las sesiones OAuth de `auth.$ .tsx / webhooks.*` también viven en Neon (tabla `Session`); no se ha dibujado la flecha para no cruzar la nota de Render.

Cómo viaja un punto de recogida hasta el carrito

Ocho pasos, siempre a través del mismo camino — el bloque de carrito nunca llama directamente al backend.

#	QUIÉN ACTÚA	QUÉ HACE
1	Bloque de carrito (JS)	Pide <code>GET /apps/pickup-points/points?postalCode=...</code> al dominio de la propia tienda — mismo origen, sin CORS.
2	App Proxy de Shopify	Reenvía la petición, firmada con HMAC, a <code><backend>/proxy/points</code> .
3	<code>proxy.points.tsx</code>	Verifica la firma con <code>authenticate.public.appProxy</code> y extrae <code>shop</code> , <code>postalCode</code> , <code>city</code> .
4	<code>pickupPoints.server.ts</code>	Busca en Postgres si esa tienda tiene configurada una URL de servicio real (<code>ShopSettings</code>).
5	Servicio externo o datos DEMO	Si hay URL, la llama y normaliza la respuesta; si no — o si falla —, usa <code>pickupPoints.mock.ts</code> (10 puntos fijos).
6	<code>geo.server.ts</code>	Geolocaliza el CP/ciudad del cliente (centroide DEMO de provincia) y ordena los puntos por distancia.
7	Bloque de carrito (JS)	Pinta la lista o el mapa (Leaflet + OpenStreetMap) con el JSON recibido.
8	Bloque de carrito (JS)	Al elegir un punto, llama a <code>/cart/update.js</code> (API propia de Shopify) y lo guarda como atributo del pedido — sin pasar por el backend.

Componentes

COMPONENTE	DÓNDE VIVE	RESPONSABILIDAD	ESTADO
Theme App Extension	Tienda (tema)	Bloque de carrito: <code>pickup-point-picker.liquid/.js/.css</code> . Modal, vista lista, vista mapa.	sin probar
App Proxy	Shopify	Reenvía <code>/apps/pickup-points/*</code> al backend con firma HMAC. Configurado en el <code>.toml</code> .	infra Shopify
Admin embebido	Shopify (iframe)	<code>app.tsx</code> + <code>app._index.tsx</code> : formulario de ajustes con Polaris.	sin probar
Backend Remix	Render (Node)	OAuth, proxy de puntos, ajustes, webhook de desinstalación.	sin desplegar
Base de datos	Neon (Postgres)	Sesiones OAuth (<code>Session</code>) y ajustes por tienda (<code>ShopSettings</code>).	migración pendiente
Servicio de puntos	Externo (agencia)	Contrato aún no definido; hoy no existe URL configurada.	no existe

Modelo de datos (Prisma, en Neon)

Session Requerida por <code>@shopify/shopify-app-session-storage-prisma</code> para persistir el token OAuth de cada tienda instalada (online y offline). <code>id, shop, accessToken, scope, isOnline, expires...</code>	ShopSettings Configuración propia de la app, una fila por tienda instalada. <code>shop, pickupApiUrl, pickupApiKey, defaultView ("list" "map")</code>
---	---

Configuración y despliegue

PIEZA	DETALLE
Config de la app	<code>shopify.app.test-plugin.toml</code> — único fichero de config activo (organización Datadec S.A. en Partners). <code>access_scopes</code> vacío a propósito: el MVP no lee ni escribe datos de la tienda.
Distribución	<code>AppDistribution.AppStore</code> en <code>shopify.server.ts</code> : pensada para que la instalen terceros, por enlace personalizado o App Store.
Hosting backend	Blueprint <code>render.yaml</code> : un servicio Node, plan free, <code>rootDir: web</code> . El build ejecuta <code>prisma generate && prisma migrate deploy && npm run build</code> .
Base de datos	Neon Postgres: <code>DATABASE_URL</code> (pooler, la usa la app en marcha) y <code>DIRECT_DATABASE_URL</code> (conexión directa, la usa <code>prisma migrate</code>).
Variables de entorno	<code>SHOPIFY_API_KEY/SECRET</code> , <code>SCOPES</code> , <code>SHOPIFY_APP_URL</code> , <code>DATABASE_URL</code> , <code>DIRECT_DATABASE_URL</code> , y opcionalmente <code>PICKUP_POINTS_API_URL/API_KEY</code> — ninguna la genera Render, hay que rellenarlas a mano.

Los tres bloqueos, y qué hace falta para cada uno

Los tres son independientes entre sí: se pueden resolver en cualquier orden.

#	QUÉ FRENA	QUÉ HACE FALTA EXACTAMENTE
1	Servicio web de la agencia bloqueado	El contrato real (endpoint, campos, autenticación) todavía no existe. Mientras tanto <code>pickupPoints.server.ts</code> sirve el listado fijo de <code>pickupPoints.mock.ts</code> . En cuanto la agencia dé la URL, solo hay que ajustar <code>normalizeExternalResponse</code> — el resto de la app no cambia.
2	Despliegue en Render bloqueado	Faltan la cuenta de Neon y el blueprint de Render por ejecutar, y rellenar a mano <code>SHOPIFY_API_KEY/SECRET</code> , <code>DATABASE_URL</code> , <code>SHOPIFY_APP_URL</code> . Hasta entonces, <code>shopify.app.test-plugin.toml</code> sigue con tres <code>REPLACE-WITH-RENDER-URL</code> (<code>application_url</code> , <code>redirect_urls</code> , <code>app_proxy.url</code>) que hay que sustituir y publicar con <code>npm run deploy</code> .
3	Migración inicial de Prisma bloqueado	<code>web/prisma/migrations</code> solo tiene el <code>migration_lock.toml</code> : nunca se ha ejecutado <code>npx prisma migrate dev --name init</code> . Sin esa migración, ni el entorno local arranca contra una base de datos real (usa SQLite de <code>.env</code> como respaldo) ni el <code>buildCommand</code> de Render tiene nada que desplegar con <code>migrate deploy</code> .

El dato que conviene no pasar por alto

El proyecto sí ha llegado a ejecutarse una vez: `.shopify/project.json` y el `dev-bundle` generado muestran que `shopify app dev` se corrió y quedó vinculado a la tienda de desarrollo `datadec-dev-pickup.myshopify.com`. Eso es lo único verificado por una herramienta hasta la fecha; no hay evidencia de que el bloque se haya abierto en un carrito real ni de que exista ningún test automático (no hay carpeta de tests ni integración continua en el repositorio).

Lo que ya está escrito y encaja con el diseño

- Los cuatro componentes (extensión, proxy, admin, servicios) y su forma de hablarse.
- El contrato interno `PickupPoint` y su normalización desde un servicio externo.
- El fallback a datos DEMO ante fallo o ausencia del servicio real.
- El cálculo de distancia y orden (`haversineKm` + geocodificador DEMO).
- El guardado del punto elegido como atributo del pedido, visible en Admin y email.

Qué falta para afirmar esto con más seguridad

- Ejecutar `npx prisma migrate dev` y levantar la app contra una base real.
- Abrir el bloque en el carrito de la tienda de desarrollo ya vinculada y probar lista, mapa y guardado.
- Un primer despliegue en Render con URLs definitivas, para probar OAuth de extremo a extremo.
- El contrato del servicio web de la agencia de transporte, para dejar de depender del mock.